PR Rechnernetze und Netzwerkprogrammierung 621.801/621.802

Institut für Informationstechnologie (ITEC) · WS 2026/2027

Übungsblatt 01

Python-Grundlagen für die Netzwerkprogrammierung

Ziel des Übungsblattes. In den folgenden Übungen werden die Python-Grundlagen erarbeitet, die im weiteren Verlauf der Lehrveranstaltung für die Netzwerkprogrammierung benötigt werden. Der Schwerpunkt liegt nicht auf dem Erlernen der gesamten Programmiersprache Python, sondern auf dem sicheren Umgang mit Dateien, Zeichenketten und Bytes, binären Datenstrukturen sowie nebenläufigen Programmen. Diese Konzepte werden in späteren Übungen direkt für die Implementierung von Netzwerkprotokollen und Client-/Server-Anwendungen verwendet.

Programmiersprache. Verwenden Sie Python 3. Die Aufgaben sollen mit der Python-Standardbibliothek gelöst werden; zusätzliche Pakete sind für dieses Übungsblatt nicht erforderlich.

Ü 1.1 Python-Umgebung und grundlegende Sprachkonstrukte

Machen Sie sich mit der für die Lehrveranstaltung verwendeten Python-Umgebung vertraut.
Verwenden Sie nach Möglichkeit eine virtuelle Umgebung (virtual environment), damit die Python-Umgebung unabhängig von anderen Projekten auf Ihrem Rechner bleibt.
Erstellen Sie eine virtuelle Umgebung und aktivieren Sie diese. Je nach Betriebssystem können die Befehle leicht abweichen.

```
mkdir rechnernetze && cd rechnernetze
python3 -m venv .venv
source .venv/bin/activate
python --version
```

Schreiben Sie anschließend ein Python-Programm hello.py, das Ihren Namen, die verwendete Python-Version und das aktuelle Arbeitsverzeichnis ausgibt.
Machen Sie sich dabei insbesondere mit folgenden Python-Konzepten vertraut:

- Variablen und grundlegende Datentypen
- if/elif/else
- for- und while-Schleifen
- Funktionen und Rückgabewerte
- Listen und Dictionaries
- Importieren von Modulen
- Kommandozeilenargumente

Ü 1.2 Text, Zeichenketten und Bytes

Bei der Netzwerkprogrammierung werden Daten letztendlich als Folge von Bytes übertragen. Python unterscheidet jedoch zwischen Zeichenketten (str) und Bytefolgen (bytes). Machen Sie sich mit diesem Unterschied vertraut.
Erstellen Sie ein Programm, das eine Zeichenkette einliest und diese zunächst als UTF-8 in Bytes kodiert. Geben Sie anschließend sowohl die Zeichenkette als auch die Bytefolge aus.

```
text = "Hallo Netzwerk!"
data = text.encode("utf-8")
```

```
print(text)
print(data)
print(len(text))
print(len(data))
```

Erweitern Sie das Programm, dass die Bytefolge wieder in eine Zeichenkette dekodiert wird.
Beantworten Sie folgende Fragen:

- Was ist der Unterschied zwischen str und bytes?
- Warum kann len(text) einen anderen Wert liefern als len(text.encode("utf-8"))?
- Was passiert, wenn eine Bytefolge mit der falschen Zeichenkodierung dekodiert wird?
- Warum ist die Unterscheidung zwischen Zeichenketten und Bytes für Netzwerkprogramme wichtig?

Ü 1.3 Datei-I/O und Kommandozeilenparameter

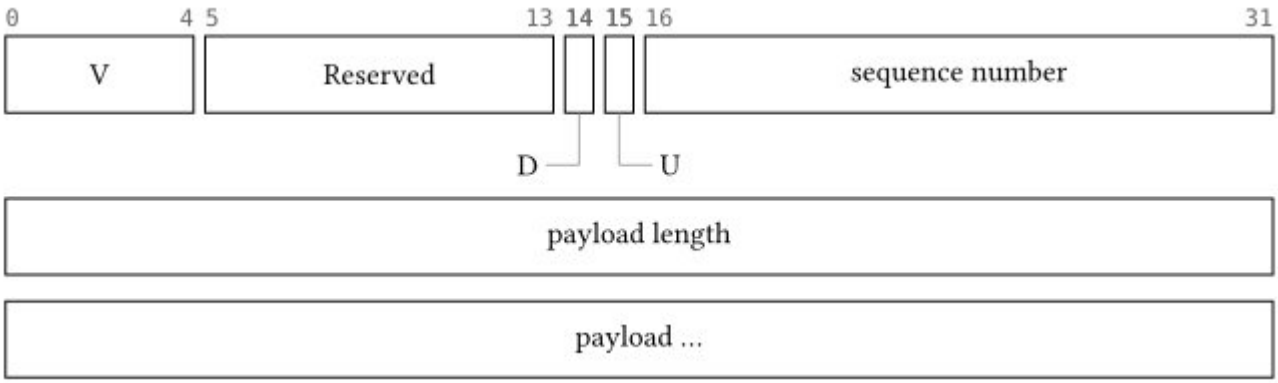
Schreiben Sie ein Python-Programm convert.py, welches den Namen einer Eingabedatei und den Namen einer Ausgabedatei als Kommandozeilenparameter erhält.
Die Eingabedatei enthält einen UTF-8-kodierten Text. Lesen Sie die Datei ein und schreiben Sie den Inhalt in eine neue UTF-8-kodierte Datei. Zeilenumbrüche sollen dabei unabhängig vom auf dem Betriebssystem verwendeten Zeilenende korrekt verarbeitet werden.
Das Programm soll insbesondere folgende Fehlerfälle geeignet behandeln:

- fehlende Kommandozeilenparameter
- nicht vorhandene Eingabedatei
- nicht lesbare Eingabedatei
- nicht beschreibbare Ausgabedatei

Verwenden Sie für die Dateiverarbeitung die Python-Standardbibliothek und achten Sie auf einen sauberen Umgang mit geöffneten Dateien.

Ü 1.4 Binäre Daten und Protokollnachrichten

Netzwerkprotokolle übertragen häufig nicht nur Text, sondern Nachrichten, die aus mehreren Feldern unterschiedlicher Größe bestehen. Die einzelnen Felder werden dabei als Bytes übertragen. Python stellt für die Abbildung solcher Datenstrukturen unter anderem das Modul struct zur Verfügung.
Gegeben sei folgendes Nachrichtenformat für ein einfaches Protokoll:

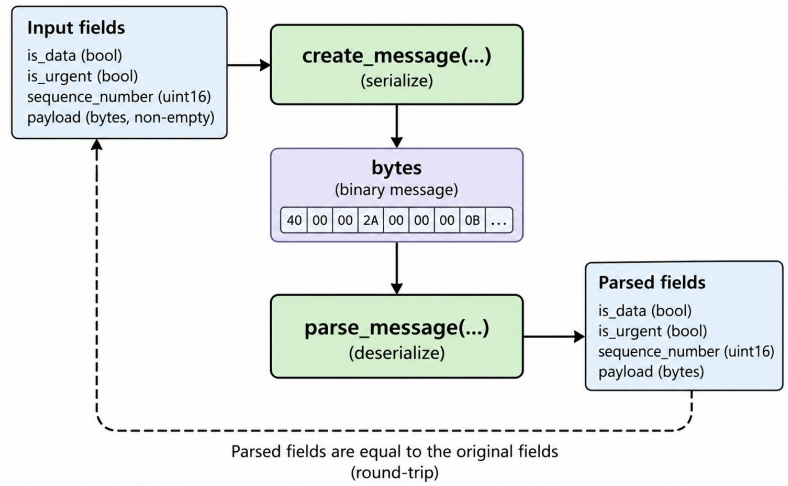


Die Nachricht besteht aus einem Versionsfeld V mit 5 Bit (fixer Wert 2), 9 reservierten Bits (immer 0), zwei Flags D und U, einer 16 Bit großen Sequenznummer und einer 32 Bit großen Payload-Länge. Anschließend folgt die eigentliche Nutzlast variabler Länge.
Implementieren Sie eine Funktion create_message(is_data, is_urgent, sequence_number, payload), die eine solche Nachricht als bytes-Objekt erzeugt. Sequenznummer und Payload-Länge sollen in Network Byte Order (Big Endian) gespeichert werden.

Die Funktion soll fehlerhafte Eingaben erkennen. Unter anderem gelten folgende Regeln:

- Die Versionsnummer muss immer 2 sein.
- Die reservierten Bits müssen 0 sein.
- Die Sequenznummer muss im Bereich einer positiven 16-Bit-Ganzzahl liegen.
- Die Payload darf nicht leer sein.
- Die tatsächlich gespeicherte Payload-Länge muss mit der Länge der Payload übereinstimmen.

Testen Sie Ihre Implementierung mit gültigen und ungültigen Eingaben.
Schreiben Sie außerdem eine zweite Funktion parse_message(message), welche die Felder wieder aus der Bytefolge extrahiert. Überprüfen Sie insbesondere, dass für eine Nachricht gilt:



Hinweis: Verwenden Sie für die Binärkodierung das Python-Modul struct. Achten Sie insbesondere auf die Wahl geeigneter Formatzeichen und auf die Byte-Reihenfolge.

Ü 1.5 Nebenläufigkeit mit Threads

Netzwerkserver müssen häufig mehrere Aufgaben gleichzeitig bearbeiten. Python stellt dafür unter anderem das Modul threading zur Verfügung.
Implementieren Sie ein Programm, das eine Liste von Gleitkommazahlen aus einer Datei liest und eine rechenintensive Funktion auf den Werten ausführt. Die Berechnung soll auf n Threads aufgeteilt werden. Jeder Thread bearbeitet einen Teil des Arrays. Die Länge des Arrays muss dabei nicht ohne Rest durch n teilbar sein.
Die eigentliche Berechnung können Sie selbst wählen. Sie sollte genügend Rechenaufwand verursachen, damit die parallele Aufteilung sichtbar wird.

```
def complicated_calculation(values):
    # eigene Berechnung
    ...
def worker(values, start, end, results, index):
    ...
threads = []
results = [None] * number_of_threads
# Threads erzeugen und starten
# anschließend mit join() auf deren Ende warten
```

Beantworten Sie folgende Fragen:

- Was ist der Unterschied zwischen dem Erzeugen und dem Starten eines Threads?
- Welche Funktion hat Thread.join()?
- Warum sollte jeder Thread möglichst auf einen eigenen Bereich des Arrays zugreifen?

- Welche Probleme können auftreten, wenn mehrere Threads dieselbe Variable gleichzeitig verändern?

Ü 1.6 Thread Safety und gemeinsam genutzte Daten

Erweitern Sie das Programm aus Ü 1.5 so, dass alle Threads ein gemeinsames Ergebnis erzeugen. Die Threads sollen ihre Teilergebnisse zu einer gemeinsamen Summe addieren.
Untersuchen Sie zunächst eine Variante, in der alle Threads die gemeinsame Variable direkt verändern. Führen Sie das Programm mehrfach aus und versuchen Sie, einen Unterschied zwischen dem erwarteten und dem tatsächlich erhaltenen Ergebnis zu beobachten.
Schützen Sie anschließend den Zugriff auf die gemeinsame Variable mit einem geeigneten Synchronisationsmechanismus aus der Python-Standardbibliothek.
Beantworten Sie:

- Was bedeutet Thread Safety?
- Warum kann eine Operation wie result += value problematisch sein, wenn mehrere Threads sie gleichzeitig ausführen?
- Welche Funktion hat ein Lock?
- Warum sollte ein Lock nur so lange wie notwendig gehalten werden?

Ü 1.7 Producer-Consumer

Implementieren Sie abschließend das klassische Producer-Consumer-Problem. Ein Producer erzeugt fortlaufend Integer-Werte und legt diese in einen gemeinsam genutzten Container. Ein oder mehrere Consumer entnehmen Werte aus diesem Container und berechnen beispielsweise die Summe der entnommenen Werte.

Anmerkung:
Das **Producer-Consumer-Konzept** trennt die Erzeugung von Daten oder Aufgaben von deren Verarbeitung. **Producer** erzeugen Daten und legen sie in einer gemeinsamen **Warteschlange (Queue)** ab. **Consumer** entnehmen die Daten aus der Queue und verarbeiten sie. Die Queue dient dabei als Puffer und ermöglicht es, dass Producer und Consumer unabhängig voneinander mit unterschiedlicher Geschwindigkeit arbeiten können.

Verwenden Sie hierfür `queue.Queue` aus der Python-Standardbibliothek. Die Queue soll eine begrenzte Größe besitzen, sodass der Producer gegebenenfalls warten muss, wenn der Container voll ist.
Ihre Implementierung soll mindestens folgende Komponenten enthalten:

- einen Producer-Thread
- einen Consumer-Thread
- eine gemeinsam verwendete Queue
- eine Möglichkeit, das Ende der Produktion zu signalisieren
- eine Ausgabe der vom Consumer berechneten Ergebnisse

Beantworten Sie anschließend:

- Warum ist queue.Queue für diese Aufgabe geeigneter als eine normale Liste?
- Wie wird verhindert, dass der Consumer auf eine leere Queue zugreift?
- Wie wird verhindert, dass der Producer eine bereits volle Queue weiter befüllt?
- Wie kann der Consumer erkennen, dass der Producer keine weiteren Werte erzeugen wird?